VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

FACULTY OF COMPUTER SCIENCE

_____ * _____

Laboratory Report

LAB 2

Database-Driven Web Development, CRUD Operations, and User Authentication

Teacher: NGUYỄN ĐỨC THÁI

Class: CC01

—o0o—

Huỳnh Đức Phát (2452940)

T3 — Medical Clinic Website

Contents

1. Project scope and assessment coverage 3
2. Environment, setup and source structure 4
3. Relational database and sample data 5
3.1 Relational model 6
4. Data dictionary and integrity rules 7
4.1 Request, audit and rate-limit records 8
5. PHP/MySQL connection and error handling 9
6. Public website and dynamic records 10
7. Create and read a medical service 11
8. Update, synchronize and publish 12
9. Intentional deletion and protected history 13
9.1 Referenced deletion and service details 14
10. User registration and patient account 15
11. Login, sessions and logout 16
12. Preference cookie and persistence 17
13. Server-side authorization 18
14. Appointment form and saved confirmation 19
14.1 Administrator-to-patient synchronization 20
14.2 Patient-to-administrator synchronization 21
15. Administration overview and patients 22
16. Responsive administration evidence 23
16.1 Responsive public and patient pages 24
16.2 Mobile registration and tested sizes 25
17. Test method and final outcomes 26
18. Validation and security test matrix 27
19. Problems encountered and resolutions 28
20. Demonstration and submission instructions 29
Appendix A. MySQL structure and password storage 30
Appendix B. Complete public page and service list 31
Appendix C. Backend check details — 1–22 32
Appendix C. Backend check details — 23–44 33
Appendix C. Backend check details — 45–60 34
Appendix D. Browser evidence and verification limits 35

1. Project scope and assessment coverage

Project: T3 — Wellnest Medical Clinic. Student: Huỳnh Đức Phát, 2452940, class CC01. The main business entity is the medical service. This Lab 2 continues the Lab 1 website with persistent MySQL data, service CRUD, user authentication, sessions, a preference cookie and server-side role authorization. It remains the foundation for Lab 3.

This report maps the supplied seven tasks and 100-point rubric to implementation files and repeatable evidence. Coverage is a self-assessment of implemented requirements, not an awarded grade. Screenshots are captured from the actual local application and phpMyAdmin; the relational diagram is generated from the documented schema.

Rubric / weight	Implementation and evidence
Structure / DB design — 10	Task 1: organized directories, six InnoDB tables, four foreign keys; sections 2–4.
PHP/MySQL / dynamic display — 15	Task 2: config/database.php, app/database.php, public catalog/details; sections 5–6.
CRUD — 20	Task 3: admin service list, create/edit form, delete confirmation; sections 7–9.
Registration / login / logout — 20	Task 4: auth/register.php, login.php, logout.php and protected dashboards; sections 10–11.
Sessions / cookies — 10	Task 5: server session and 30-day sidebar preference; sections 11–12.
Role-based access — 10	Task 6: require_login(), require_admin() and request ownership; section 13.
Validation / basic security — 10	Prepared statements, password hashing, CSRF, validation, FK/version checks and HTTP denial tests; sections 7–17.
Documentation / demonstration — 5	Task 7 and deliverables: test JSON, forty original screenshots, SQL, PDF/DOCX, README and submission ZIP; sections 17–20 and appendices.

Scope: this is an educational clinic demonstration. It does not provide clinical care, send email, collect payments or reserve a real appointment. All submitted contact information should be illustrative. Public services, admin records and patient requests share the same MySQL database.

2. Environment, setup and source structure

Runtime used for verification: XAMPP Apache, PHP 8.0.28, MariaDB 10.4.28, PDO MySQL, mbstring and PHP sessions. The application needs PHP 8.0+ and MySQL/MariaDB; it does not require Node.js or Composer to serve pages. Fonts, Bootstrap, JavaScript and photographs are bundled locally. Node/Playwright and Python are QA/report tools only.

Directory	Responsibility
public/	Public entry point; css/, js/, images/ and vendor/ assets.
pages/	Service details, authenticated appointment form and patient dashboard.
components/	Shared head, header, footer, service cards and admin layout.
config/	Central database configuration; local.example.php documents optional private overrides.
app/	Bootstrap/helpers, PDO queries, authentication, service and appointment operations.
auth/ and admin/	Registration/login/logout and role-protected administration pages.
database/	schema.sql for the schema; lab2.sql for schema plus sample data.
storage/	Writable server session storage; private runtime data is excluded from submission.
tests/ and tools/	Meaningful integration checks, report generation and submission packaging.
report/lab2/	PDF/DOCX, original screenshots, evidence index and verification results.

Installation: (1) copy medical_clinic into XAMPP htdocs; (2) enable Apache and MySQL; (3) import database/lab2.sql in phpMyAdmin; (4) if needed, copy config/local.example.php to config/local.php and supply host, port, database, user and password; (5) open http://localhost/medical_clinic/. No setup HTTP endpoint is exposed.

Default import database: wellnest_lab2. The local application uses a database account restricted to this database; private credentials are omitted from Git and the ZIP. On a fresh XAMPP installation the documented default configuration uses local root with an empty password. Sessions require write access to storage/sessions; macOS setup supports an explicit Apache daemon ACL without world-writable source permissions.

Apache .htaccess blocks app/, config/, database/, storage/, tests/, tools/, report/, submission/ and hidden paths. The bundled PHP development router applies equivalent path restrictions. Read README.md for exact URLs, optional environment variables and commands.

3. Relational database and sample data

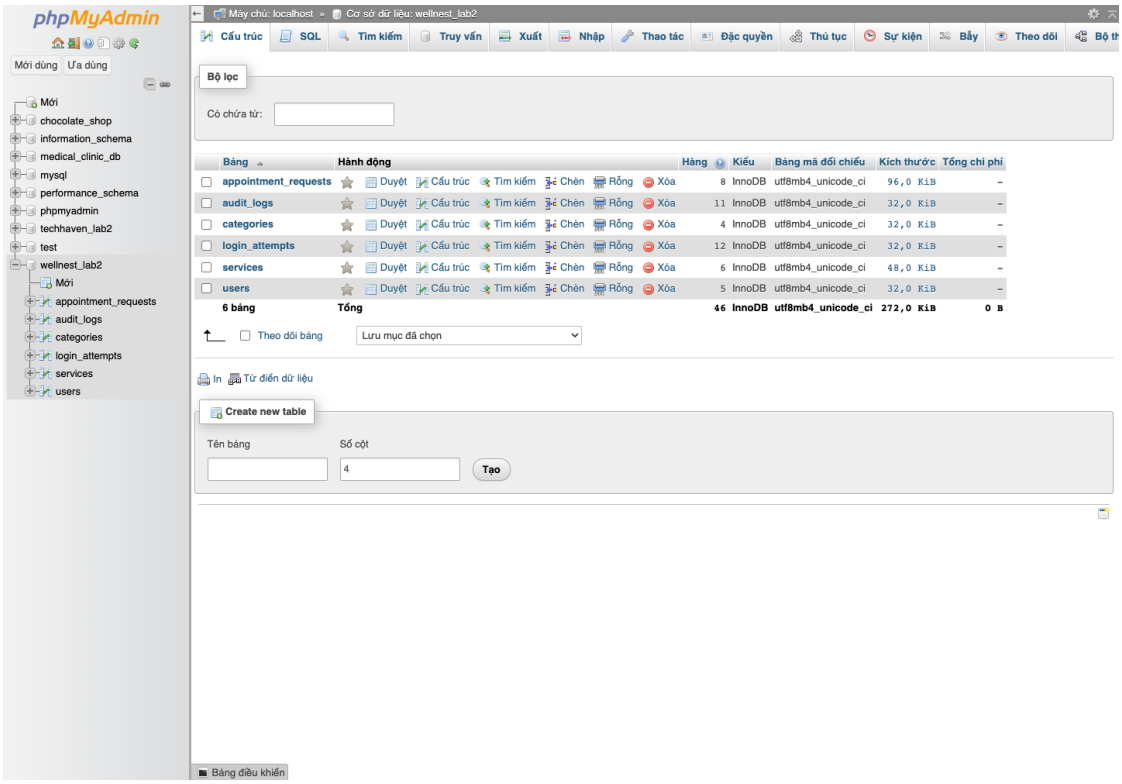


Figure 1. Actual phpMyAdmin structure view: six tables in wellnest_lab2.

All six tables use InnoDB. Database encoding is utf8mb4 with utf8mb4_unicode_ci collation, preserving Vietnamese names and enforcing case-insensitive unique email identifiers. Every table has an unsigned auto-increment primary key. SQL import creates four foreign-key relationships and relevant lookup indexes.

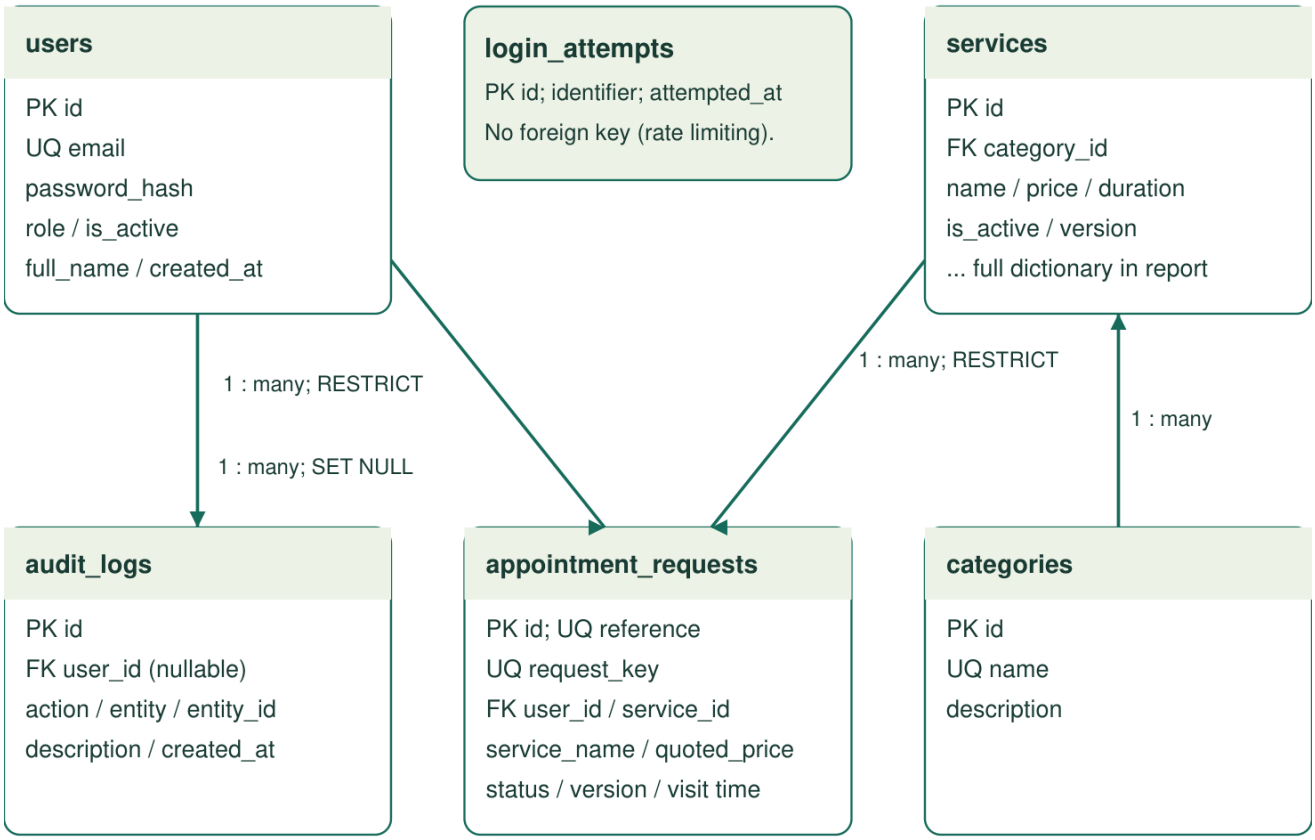
Relationship	Cardinality and deletion behavior
categories.id → services.category_id	One category to many services; ON DELETE RESTRICT.
users.id → appointment_requests.user_id	One account to many requests; ON DELETE RESTRICT.
services.id → appointment_requests.service_id	One service to many requests; ON DELETE RESTRICT.
users.id → audit_logs.user_id	One user to many audit entries; ON DELETE SET NULL.

The SQL includes four accounts (one administrator and three patients), four categories, six medical services, five illustrative requests across four statuses and a setup audit entry. Fresh-import counts are checked in a disposable test database. Browser screenshots can contain explicitly named QA records created during the demonstration; these are cleaned after testing.

The unique email key prevents duplicate accounts. Unique reference and request_key fields prevent duplicate request identifiers. Price, duration and active-status CHECK constraints complement PHP validation on MariaDB. MySQL versions that ignore CHECK constraints still receive server-side validation.

3.1 Relational model

Lab 2 relational model — key fields



PK: primary key. FK: foreign key. UQ: unique. Arrows: parent -> child.

Selected attributes shown; all fields and constraints are supplied in database/lab2.sql.

Diagram A. Schema relationship diagram generated for this report. It is a diagram, not a screenshot. All four drawn foreign-key relationships are validated against database/schema.sql by tools/generate_lab2_diagram.py. The arrows run from the parent table to its child records; selected key attributes are shown for readability.

The supporting categories table groups medical services. Users own appointment requests, and each request refers to one service. Audit entries can retain their descriptions even if their optional actor reference becomes null. Login attempts deliberately have no user foreign key because failed attempts can refer to a nonexistent account.

Full-resolution PDF and SVG versions of this diagram are supplied in report/lab2/diagrams/. The following data dictionary lists the complete field groups and the submitted SQL specifies exact constraints and defaults.

4. Data dictionary and integrity rules

Core account and service fields are listed below. PK means primary key, FK means foreign key and UQ means unique. Full DDL, defaults, indexes and exact column definitions are supplied in database/lab2.sql; the accompanying schema.sql contains the same table definitions.

Table / fields	Types and purpose
users.id / full_name / email	INT UNSIGNED PK / VARCHAR(100) / VARCHAR(190) UQ.
users.password_hash / role	VARCHAR(255) hash; ENUM(admin,patient), default patient.
users.is_active / created_at	TINYINT active flag; TIMESTAMP creation time.
categories.id / name / description	INT UNSIGNED PK; VARCHAR(60) UQ; VARCHAR(240).
services.id / category_id	INT UNSIGNED PK / INT UNSIGNED FK to categories.
services.name / price / duration	VARCHAR(100); INT UNSIGNED VND; SMALLINT UNSIGNED minutes.
services.image / alt / tag	VARCHAR(100) local asset name; VARCHAR(240) alternative text; VARCHAR(80) tagline.
services.summary / description	VARCHAR(240) card introduction; TEXT full detail.
services.includes_json / prepare	TEXT JSON array of included items; TEXT preparation instructions.
services.is_active / version	TINYINT published/draft; INT UNSIGNED optimistic version, initially 1.
services.created_at / updated_at	TIMESTAMP creation and last update.

Application limits: service names 3–100 characters, summary 10–240, description 30–6000, preparation 10–1000, price 1–99,999,999 whole VND and duration 5–240 minutes. Included items are 1–8 plain-text lines, at most 180 characters each. Images must come from the six-file local image allowlist.

A service edit increments version only when the submitted version matches the database. A stale edit therefore returns HTTP 409. A referenced service cannot be deleted because its request history still needs it; Draft is the supported way to hide it publicly while retaining history.

4.1 Request, audit and rate-limit records

Table / fields	Types and purpose
appointment_requests.id	INT UNSIGNED PK.
reference / request_key	VARCHAR(24) UQ public reference; VARCHAR(64) UQ idempotency key.
user_id / service_id	INT UNSIGNED FKs to the owning account and selected service.
service_name / quoted_price	VARCHAR(100) and INT UNSIGNED immutable submission snapshots.
full_name / email	VARCHAR(100) / VARCHAR(190) illustrative contact details.
preferred_date / preferred_time	DATE / TIME, interpreted in Ho Chi Minh City time, GMT+7.
status	ENUM(requested,confirmed,completed,cancelled), default requested.
version / created_at / updated_at	INT UNSIGNED optimistic version; two TIMESTAMP fields.
audit_logs.id / user_id	INT UNSIGNED PK; nullable FK to the actor.
audit_logs.action / entity / entity_id	VARCHAR(50) action; VARCHAR(50) entity; nullable INT UNSIGNED record identifier.
audit_logs.description / created_at	VARCHAR(255) summary and TIMESTAMP.
login_attempts.id / identifier / attempted_at	INT UNSIGNED PK; CHAR(64) SHA-256 email/IP identifier; TIMESTAMP.

Request state machine: requested → confirmed or cancelled; confirmed → completed or cancelled; completed and cancelled are terminal. Patients can cancel their own requested/confirmed requests. Only administrators can confirm or complete requests. Ownership and version checks run inside a transaction with row locking.

The request transaction reloads and locks the active service before saving its name and price. A client-supplied fee is ignored. Subsequent service edits do not rewrite historical quotations. A unique request key is retained for safe form retries; tests verify that the same form retry returns the original reference.

Registration and appointment email validation both use the database limit of 190 characters. Dates must match an ISO calendar date, fall between tomorrow and 90 days ahead and avoid Sundays. Allowed preferred times are 09:00, 10:00, 11:00, 14:00, 15:00 and 16:00. These are preferences, not reserved slots.

5. PHP/MySQL connection and error handling

config/database.php is the single configuration source. Optional local overrides and CLINIC_DB_* environment variables support different machines and isolated QA databases. app/database.php creates one PDO connection per request, selects utf8mb4, disables emulated prepares and enables exception error handling. SQL time zone is +07:00, matching PHP and the page labels.

Read path	How database data reaches the interface
Public catalog	services() joins services and categories and returns only is_active=1 records.
Service details	find_service() validates the ID and loads the same service/category row.
Admin management	The list includes drafts, filters/searches with bound parameters and paginates results.
Patient history	A WHERE user_id=? condition limits the query to the signed-in account.
Dashboard statistics	Counts and recent activity are queried from current MySQL records.

All dynamic SQL parameters use PDO prepared statements. Numeric pagination offsets are computed from validated server-side integers. No public page reseeds the database on each request. Therefore a saved admin edit persists and the next frontend request reads the updated row.

Unexpected exceptions are written to the local PHP error log. End users receive an HTTP 503 response with a short generic message. JSON clients receive {ok:false, errors:{form:...}} with the same status. No SQL statement, database name, credentials, server path or stack trace is included.

Evidence: tools/verify_error_handling.py launches a separate local PHP process with a randomly named missing database and private temporary session directory. Both HTML and JSON requests were checked for HTTP 503 and absence of sensitive details. It leaves config/local.php and the working Apache application untouched. Results: error-handling-verification.json.

Backend tests independently import lab2.sql under a temporary database name, assert PDO uses the mysql driver, verify all six tables and all four foreign keys, and check the demo password hashes. The disposable database is dropped after the run.

6. Public website and dynamic records

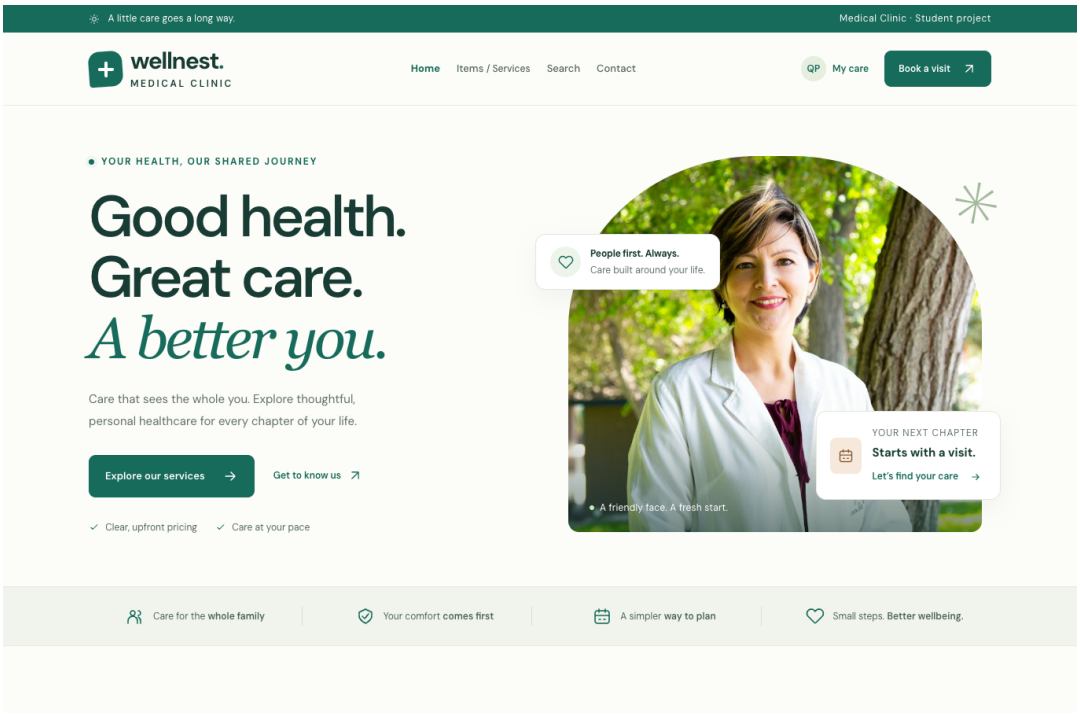


Figure 2. Public homepage served by Apache with local visual assets.

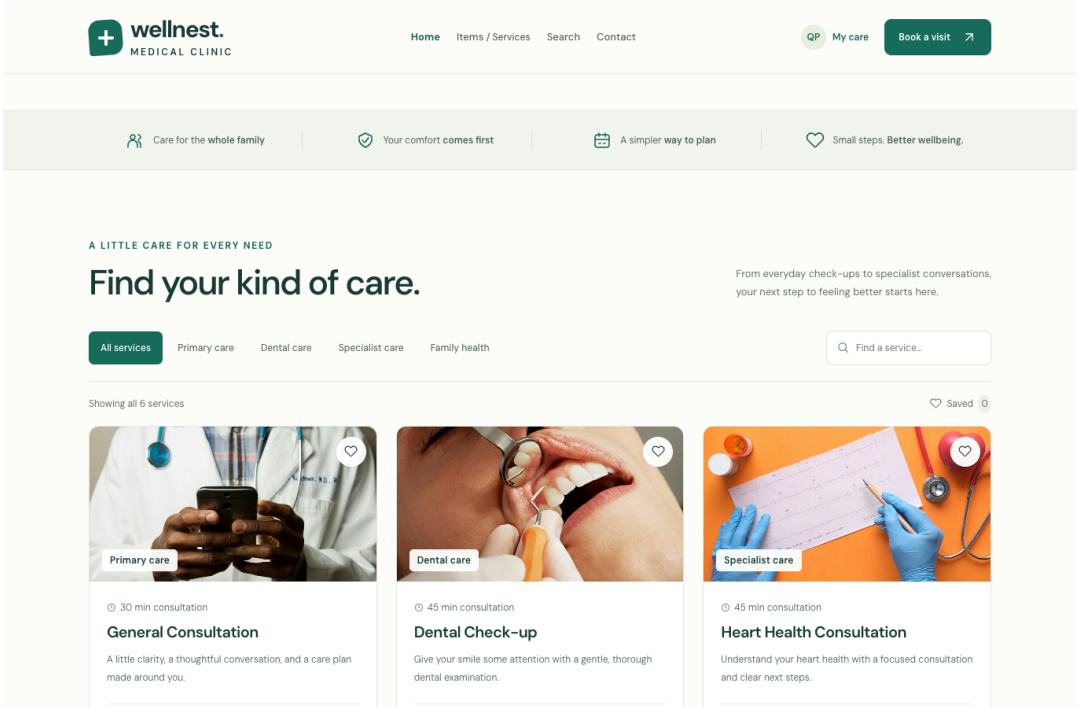


Figure 3. Medical service cards use actual joined MySQL service/category records.

Catalog search, category filters and saved-service interactions operate on the rendered database records. New IDs above the original six are supported. The saved-service list uses localStorage for a non-sensitive convenience; this is separate from the cookie requirement implemented in section 12.

7. Create and read a medical service

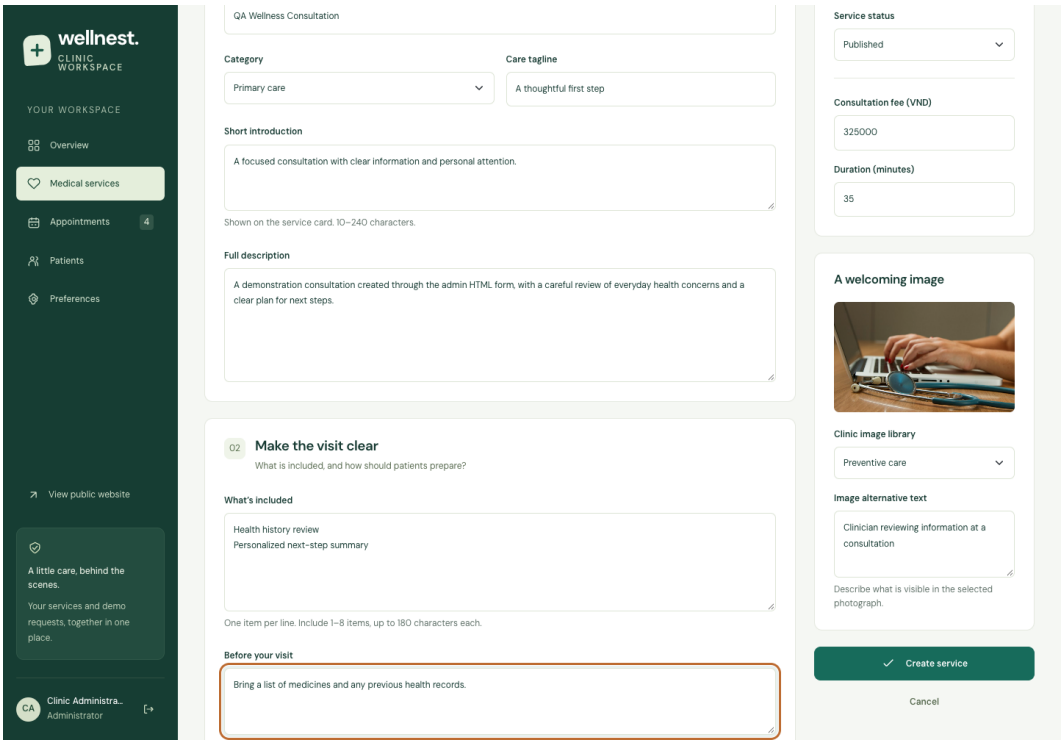


Figure 4. Administrator enters a QA service through the HTML form.

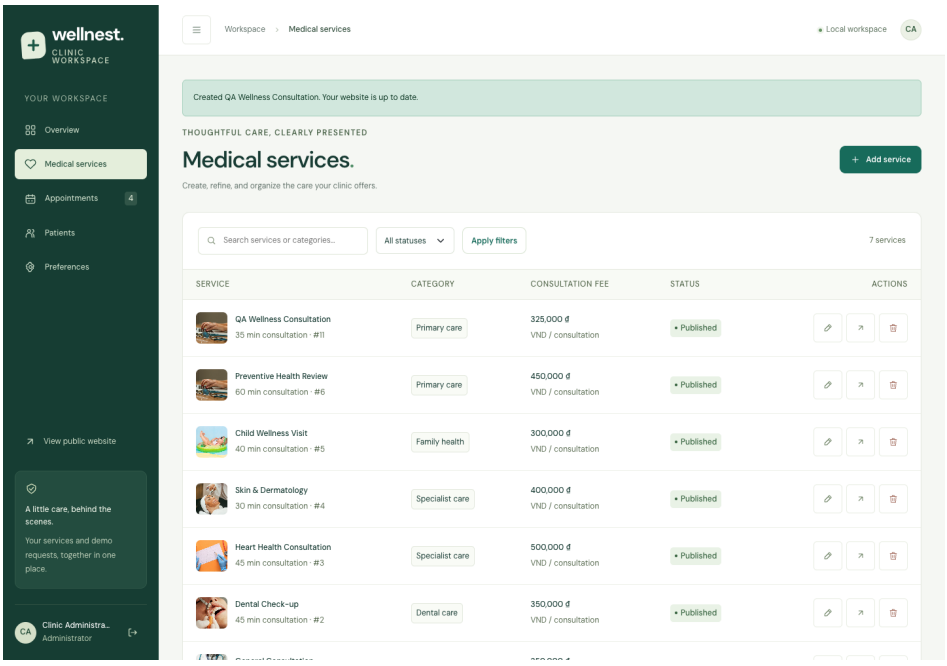


Figure 5. The successful insert appears in the admin service list.

Procedure: sign in as administrator → Medical services → Add service → complete essentials, visit details, image, fee and publication status → submit. PHP checks CSRF and every required field, then inserts the row and audit entry in a transaction. Successful submission uses POST/Redirect/GET and provides a success notice.

The browser demonstration created QA Wellness Consultation at 325,000 VND. Its public card and detail page immediately displayed the same name and fee. A forged CSRF token returned HTTP 403; a negative fee returned HTTP 422 and a clear validation message.

8. Update, synchronize and publish

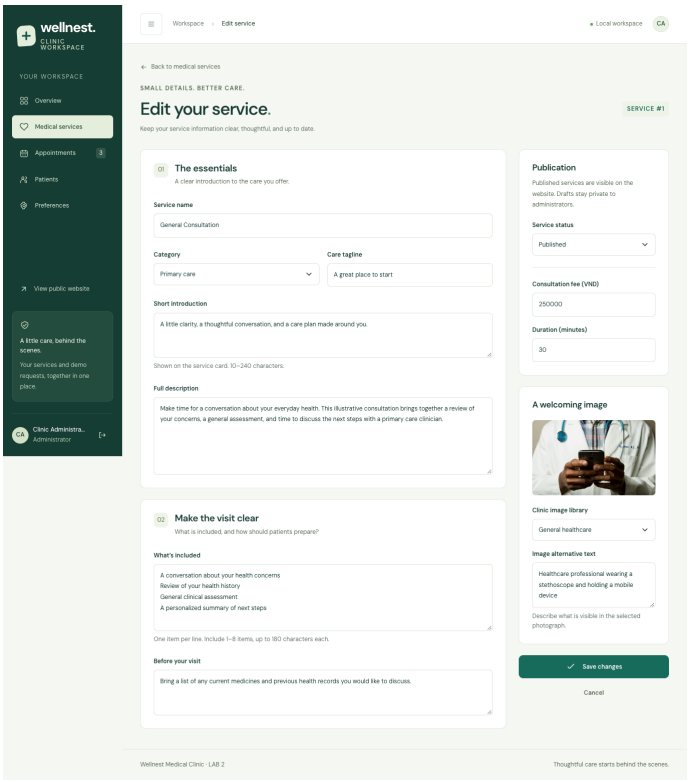


Figure 6. Edit form loads the existing database record and version.

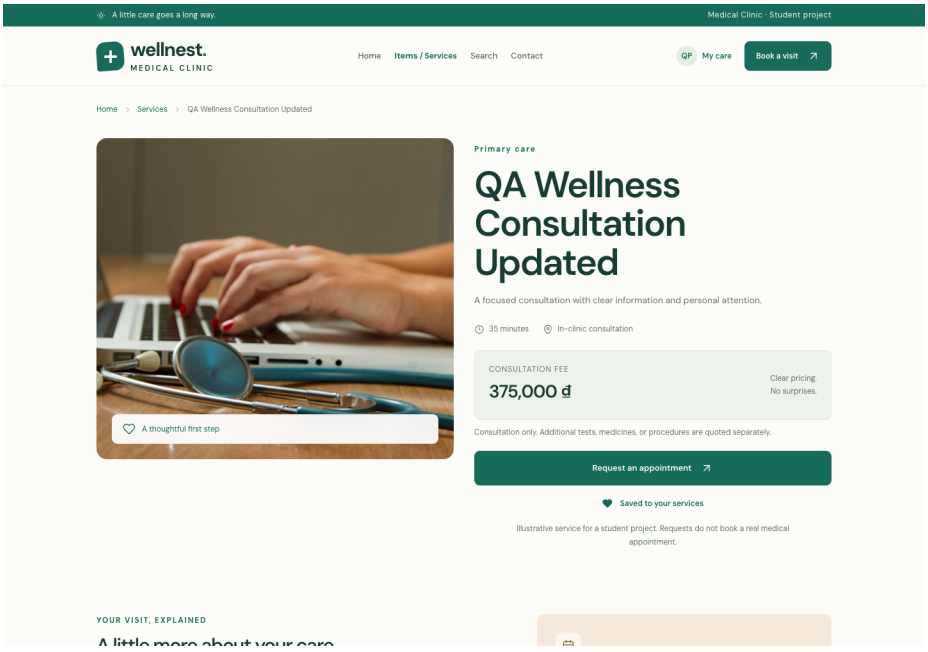


Figure 7. The same public URL displays the updated consultation fee.

Procedure: edit the QA service → change its name to QA Wellness Consultation Updated and fee to 375,000 VND → save → reload its existing public details URL. The browser checked the exact resulting name/fee. A previously captured stale form was rejected with HTTP 409.

The same record was then changed to Draft. Its public details URL returned HTTP 404 while its admin edit URL remained available. Published services are also the source for the appointment selector, keeping public discovery and new request choices consistent.

9. Intentional deletion and protected history

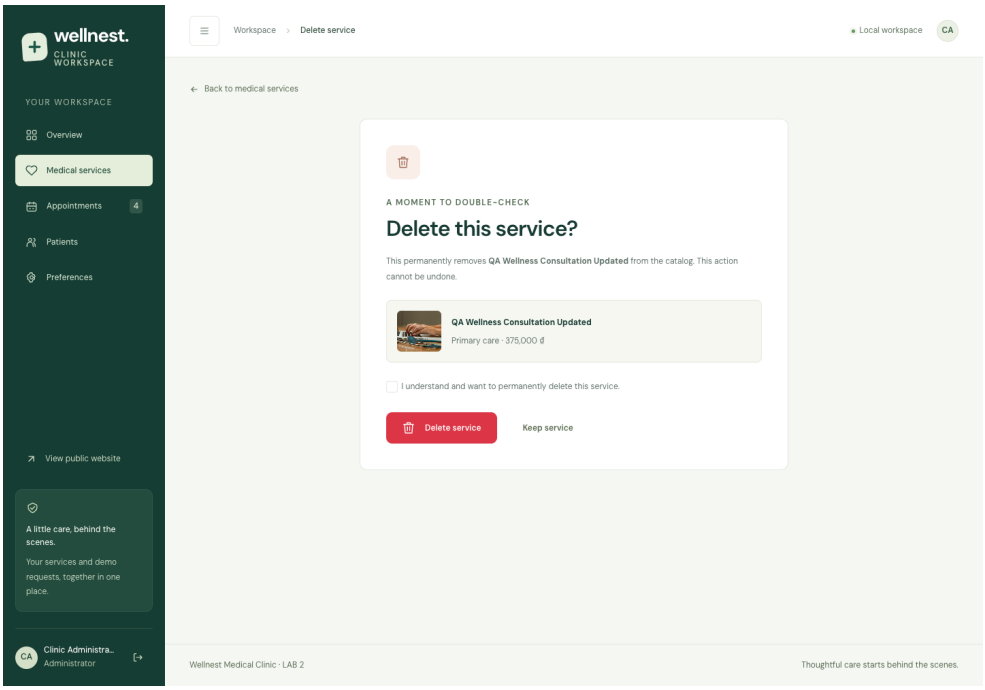


Figure 8. Deletion requires an explicit checkbox and POST submission.

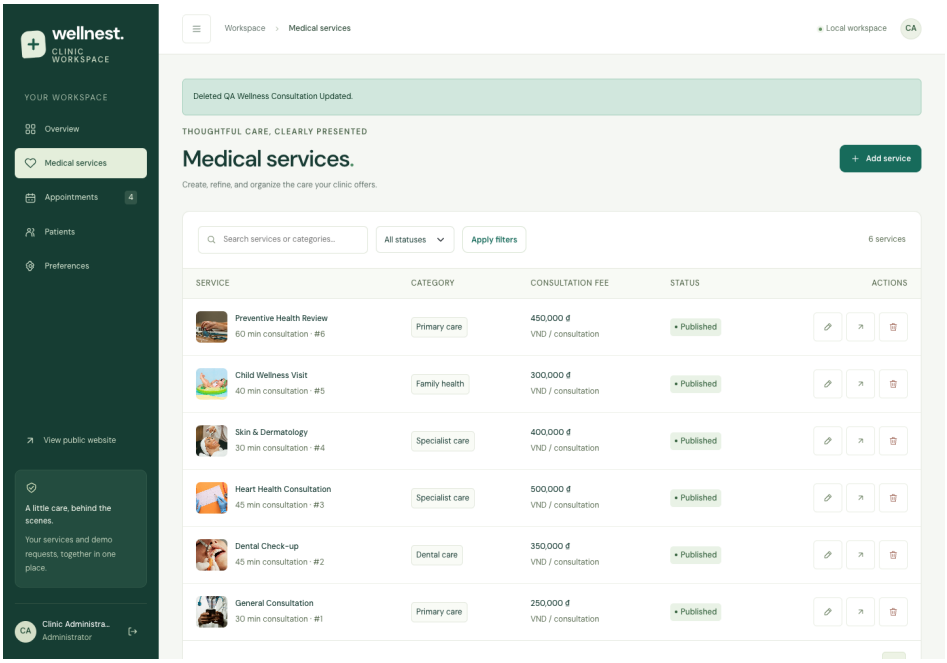


Figure 9. An unreferenced QA service is removed with success feedback.

Opening admin/service_delete.php is a read-only confirmation step. The browser verified that the record still existed after GET and that a POST without confirmation returned HTTP 422. Only a valid CSRF-protected POST with confirm_delete=1 removed the QA record; subsequent admin reads returned HTTP 404.

Deletion and its audit entry use a transaction. A database exception caused by existing appointment relationships is converted into an actionable message rather than a technical SQL error. The administrator can keep history and hide the service by choosing Draft.

9.1 Referenced deletion and service details

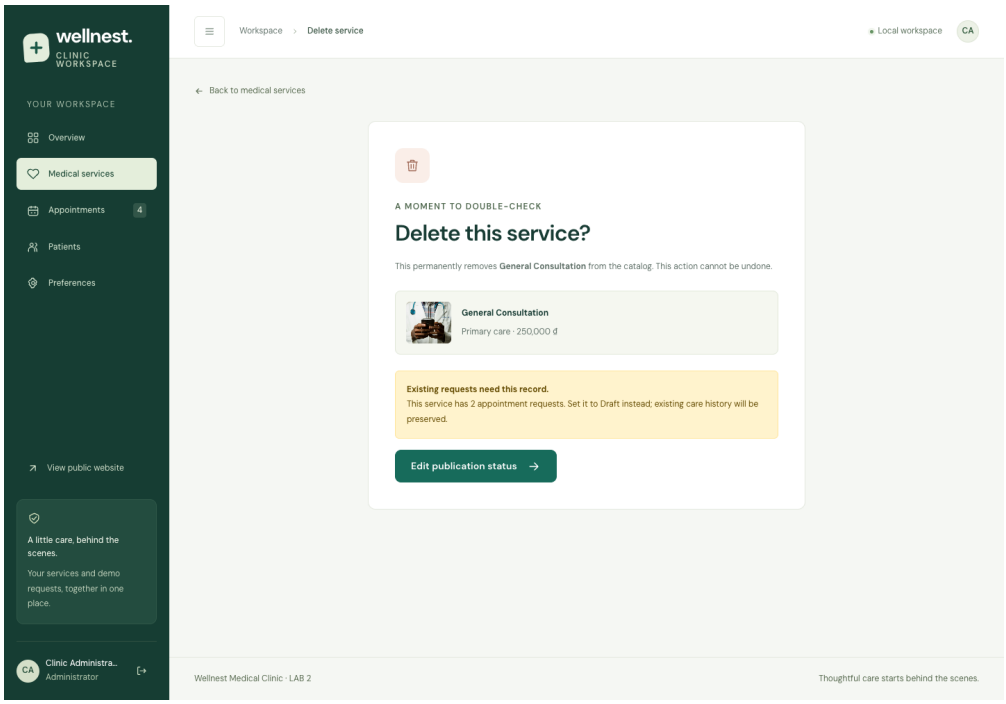


Figure 10. Existing appointment requests prevent deletion of a referenced service.

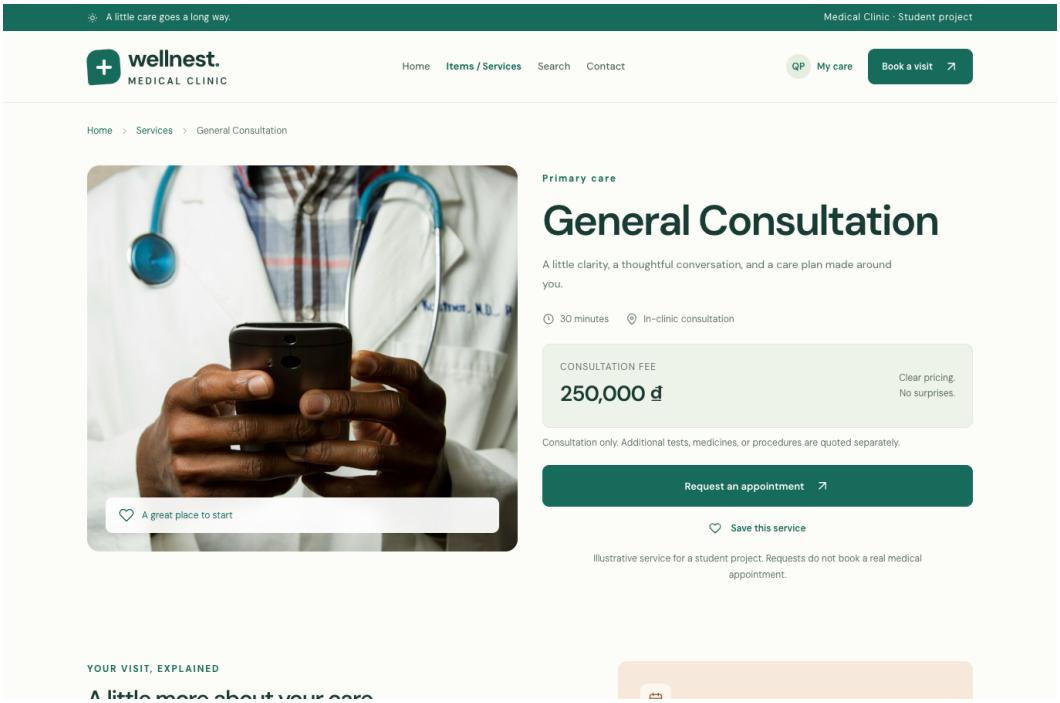


Figure 11. Public details, pricing, duration and visit preparation use the current database record.

The browser also attempted a direct delete POST against service #1. The server returned HTTP 409 and retained the record, demonstrating enforcement beyond a disabled button. Backend checks verified that later service edits preserve prior request name/price snapshots.

10. User registration and patient account

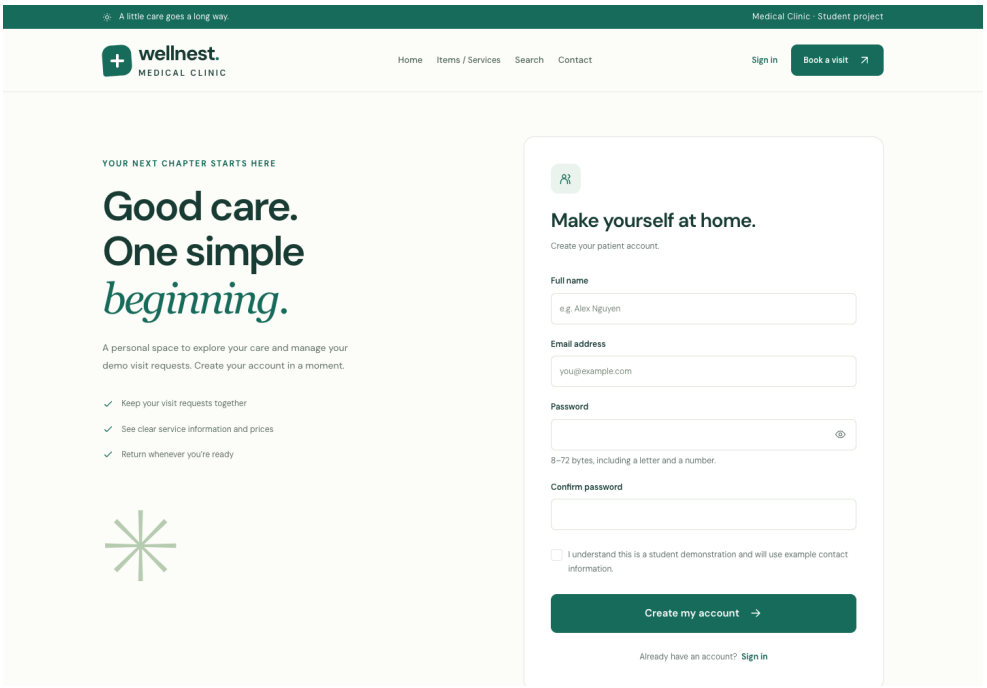


Figure 12. Registration collects name, email, password confirmation and demo acknowledgement.

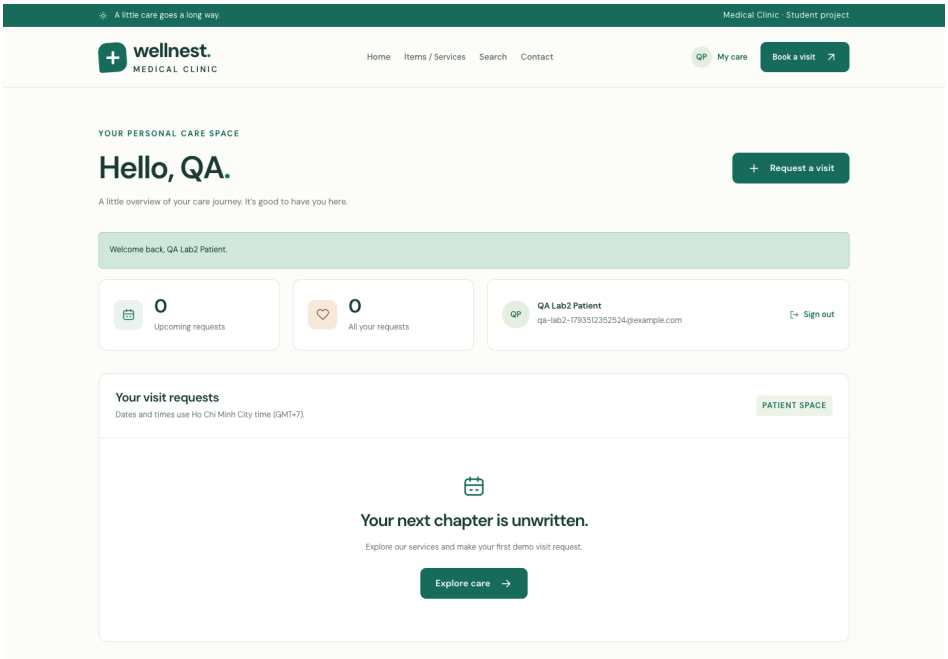


Figure 13. The newly registered account signs in to a protected patient dashboard.

Required server checks: name 2–100 characters without markup/control characters, valid email up to 190 characters, unique email, matching password confirmation and demo acknowledgement. Passwords require 8–72 bytes with a letter and a digit; null bytes are rejected. The byte limit avoids silent bcrypt truncation.

password_hash(PASSWORD_DEFAULT) stores a secure hash. password_verify() validates it during login. Database uniqueness handles duplicate submissions, including email case differences. Registration always assigns patient on the server; the browser submitted role=admin and confirmed that admin access still returned HTTP 403.

11. Login, sessions and logout

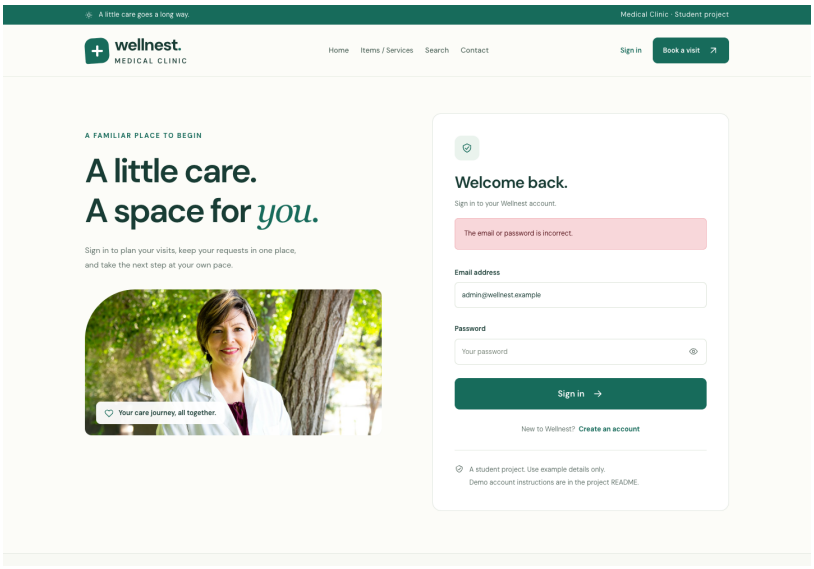


Figure 14. Invalid login returns a general credential error.

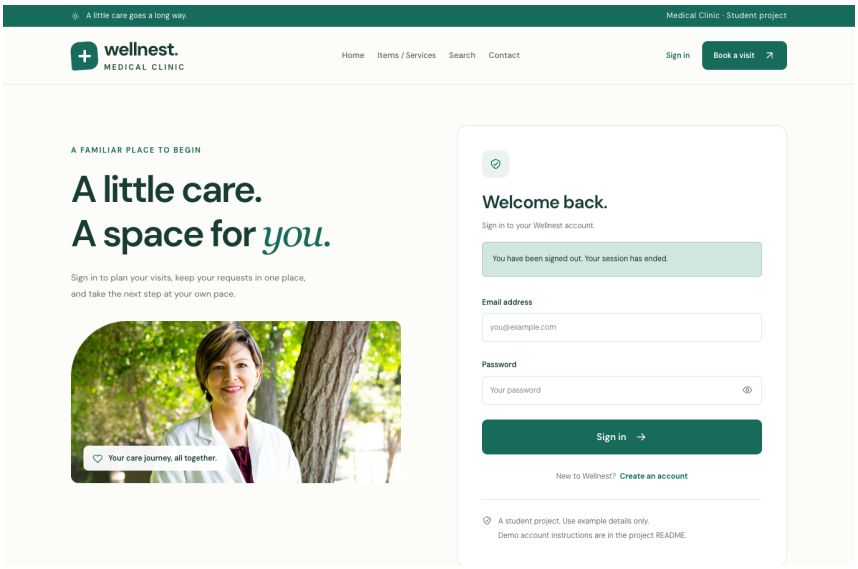


Figure 15. Logout confirms that the authenticated session has ended.

Successful login verifies an active database account, regenerates the session ID, rotates the CSRF token and stores user_id and last_activity on the server. An allowlisted next path can return the user to a protected form; external and role-inappropriate redirect targets are rejected.

The session cookie is wellnest_lab2_session, scoped to the project path with HttpOnly and SameSite=Lax. Secure is enabled under HTTPS. Its value is an opaque identifier, not a password. Authentication expires after 2 hours of inactivity; the session cookie is browser-session scoped. Browser session restoration can preserve browser-session cookies, so server inactivity expiry and explicit logout are the enforced limits.

Logout accepts POST with CSRF, clears server session data, expires the session cookie and destroys the session. Tests verified protected access is denied afterward and that replaying the pre-logout cookie cannot restore administrator access. GET logout is rejected by the route. Failed login attempts are limited to six per email/IP identifier within 15 minutes; a seventh attempt returns HTTP 429.

12. Preference cookie and persistence

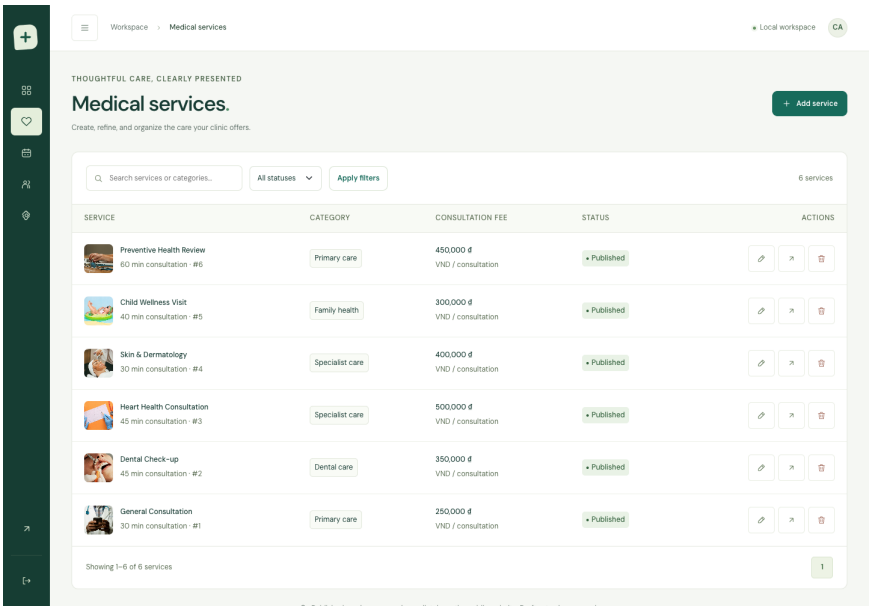


Figure 16. Collapsed desktop sidebar remains collapsed after reloading.

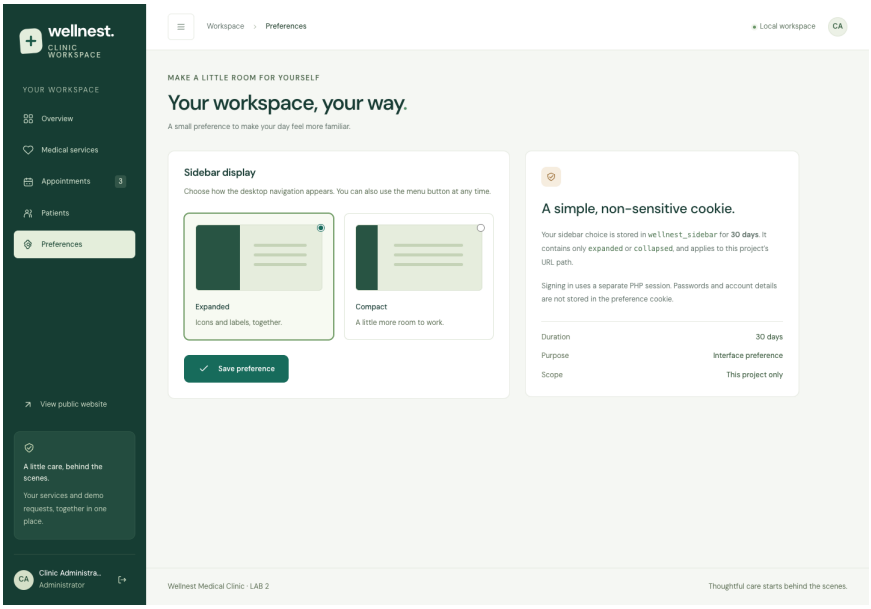


Figure 17. Preferences documents and changes the sidebar cookie.

Cookie purpose: remember the administrator's sidebar width. Name: wellnest_sidebar. Allowed values: expanded or collapsed. Lifetime: 30 days from the latest change. Path: the medical_clinic project root. SameSite=Lax; Secure is added on HTTPS. It contains no account information or authentication secret.

The desktop toggle writes the preference and PHP reads it on subsequent page renders. A normal PHP form provides the same preference update. The cookie is intentionally JavaScript-readable because the toggle writes it; the authentication cookie remains HttpOnly. Tests inspected actual cookie value, SameSite policy and expiry, and reloaded the page to verify persisted rendering.

This demonstrates both required mechanisms independently: PHP sessions maintain authentication; a non-sensitive cookie maintains an interface preference. Disabling the preference cookie does not grant any protected capability.

13. Server-side authorization

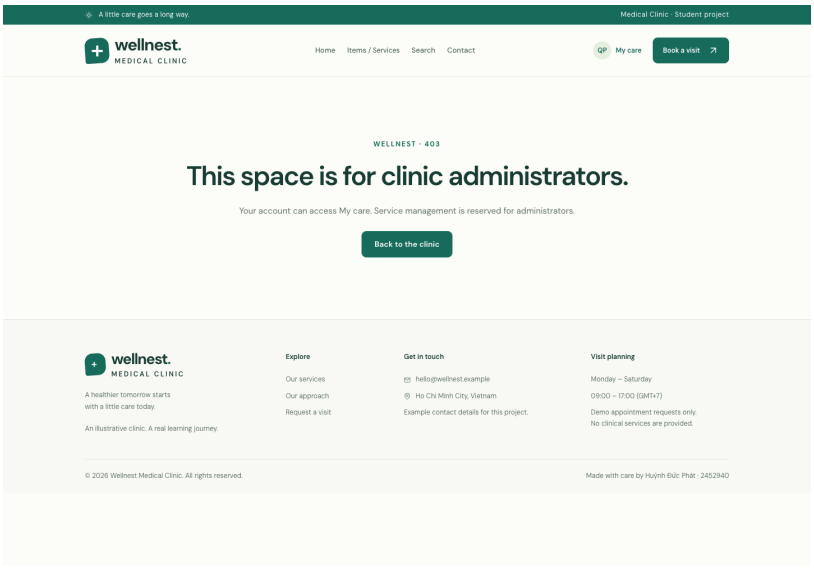


Figure 18. A signed-in patient receives HTTP 403 for administrator functionality.

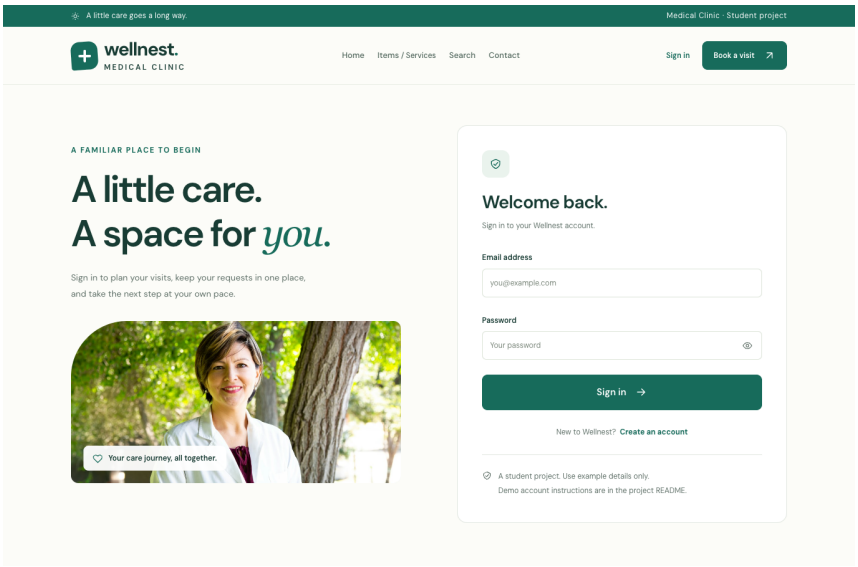


Figure 19. An unauthenticated request to a protected page returns to sign-in.

Actor	Server-enforced capability
Guest	Read public services; use registration/login. Protected HTML requests redirect to login, JSON requests return HTTP 401.
Patient	Submit requests, read their own history and cancel their own requested/confirmed records. All admin routes reject access.
Administrator	Manage services, review requests, perform allowed status changes, view patient directory and preferences.

Every admin route calls require_admin() before reading protected data or processing mutations. The browser tested all seven admin routes and a direct form POST using a patient session. Request mutations also check ownership inside the transaction. Hiding menu links is only presentation; it is not the access-control mechanism.

14. Appointment form and saved confirmation

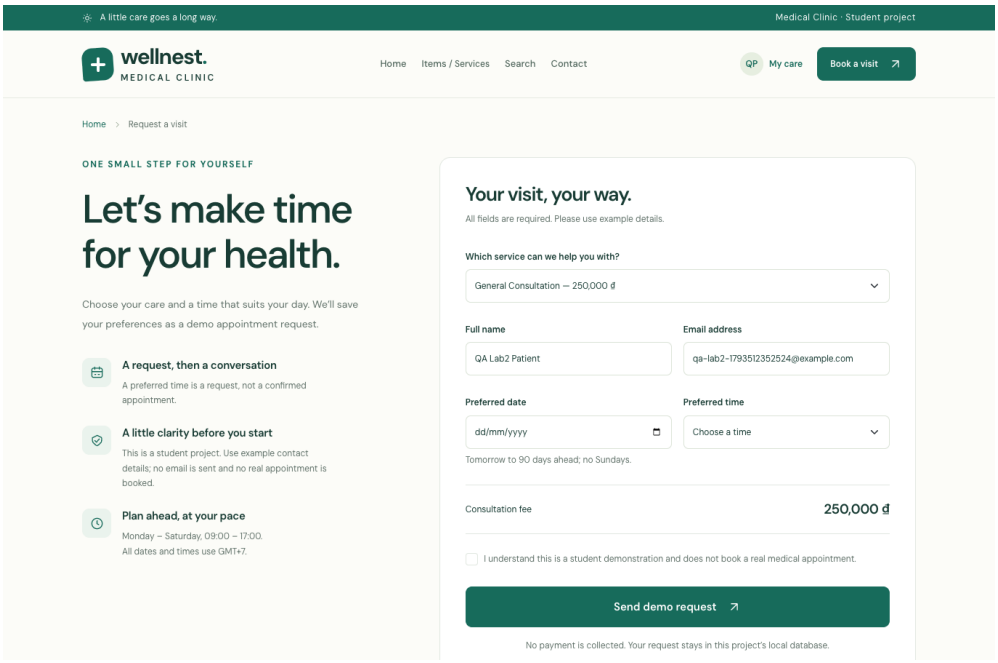


Figure 20. Authenticated request form reads published services and prices from MySQL.

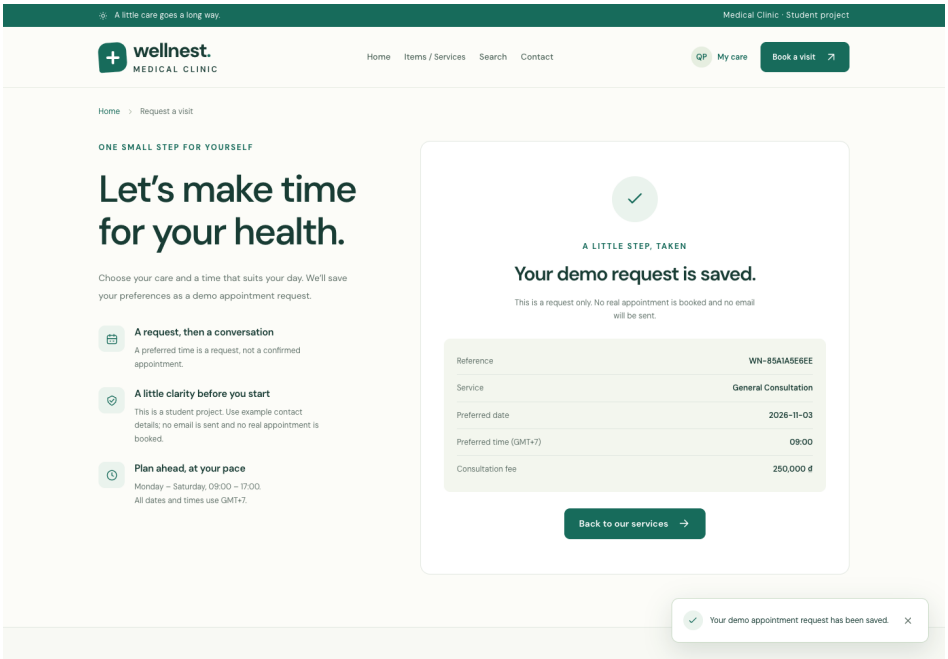


Figure 21. A saved request returns its reference, preferred time and database fee.

Procedure: sign in as patient → choose a service → choose a valid date and preferred time → acknowledge the demonstration → submit. PHP validates the input, session-bound request key and CSRF token. The service is reloaded in the transaction before the request is stored.

The browser submitted a forged quoted_price=1 during a retry. The response retained the original 250,000 VND database fee and the original request reference, proving both trusted server pricing and idempotent retries. The native PHP form also worked with JavaScript disabled and used POST/Redirect/GET.

14.1 Administrator-to-patient synchronization

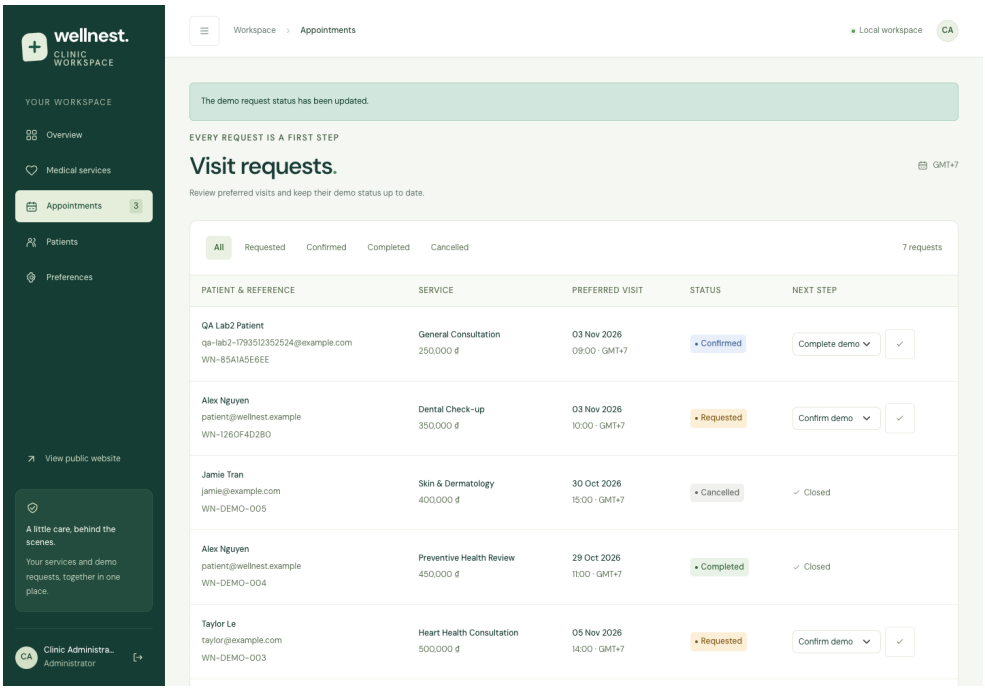


Figure 22. Administrator changes the QA request from Requested to Confirmed.

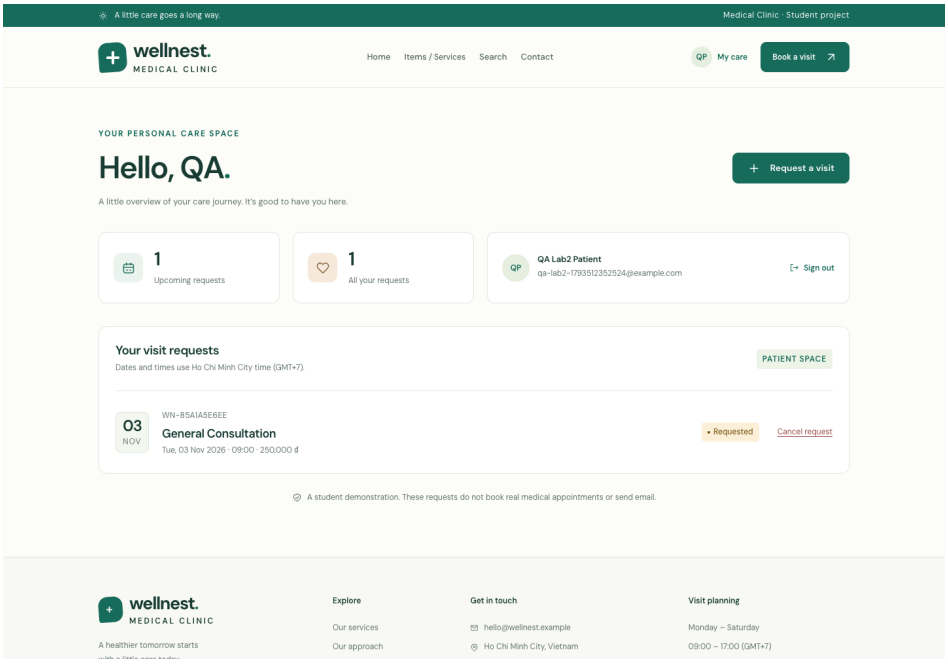


Figure 23. Patient request history is backed by the same persisted request records.

The browser used the same unique QA reference on both interfaces. After the administrator confirmed the request, the patient dashboard was reloaded and its exact row was checked for Confirmed. This demonstrates persisted synchronization across separate authenticated browser contexts.

Allowed transitions and version checks prevent stale tabs from silently overwriting status changes. These statuses describe the local demonstration request only; confirmation does not send a message or reserve an actual clinical slot.

14.2 Patient-to-administrator synchronization

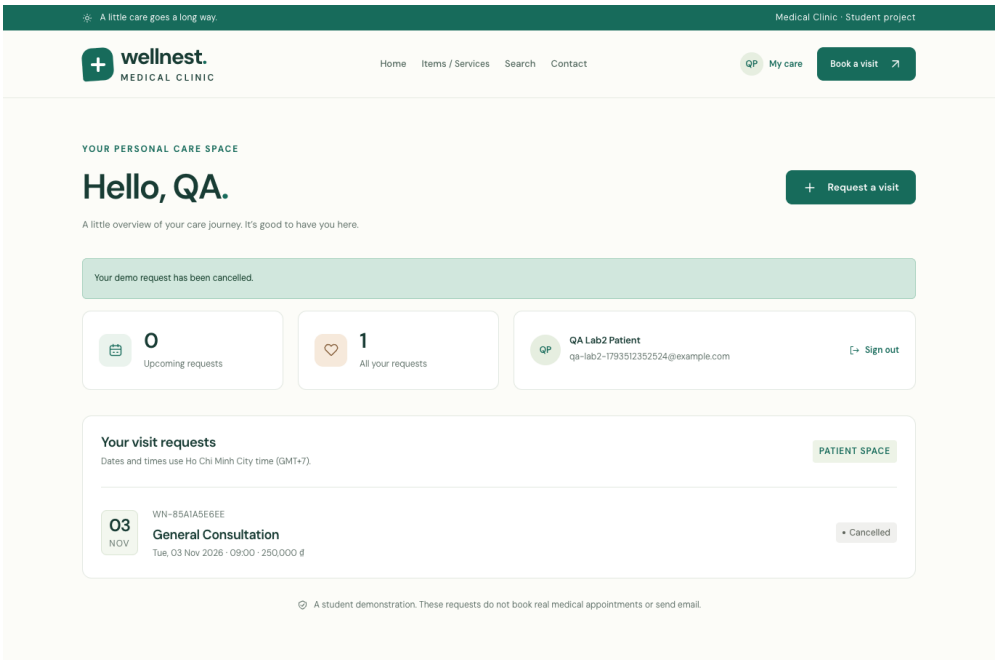


Figure 24. The owning patient intentionally cancels the confirmed QA request.

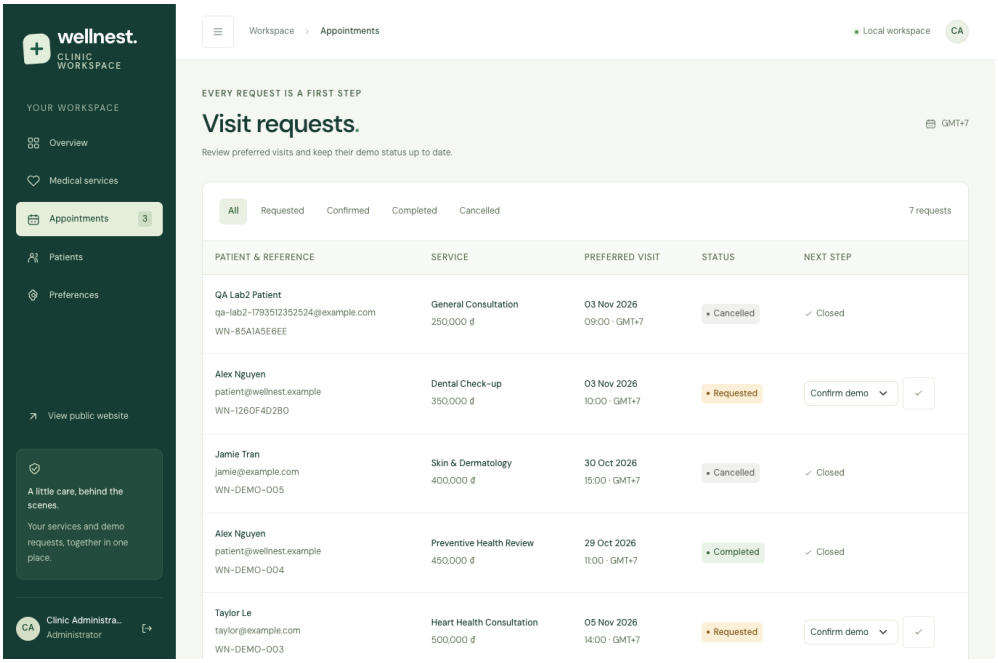


Figure 25. The same reference becomes Cancelled in the administrator list.

Procedure: return to the patient dashboard → Cancel request → confirm the dialog → reload the administrator request list. Both interfaces display Cancelled for the same database row. Backend tests additionally prove that patients cannot confirm their own requests, cancel another account's request or reopen a terminal request.

Both directions therefore use a shared persistent record, while the service name and quoted fee remain historical snapshots. Each accepted mutation appends an audit entry within the same transaction.

15. Administration overview and patients

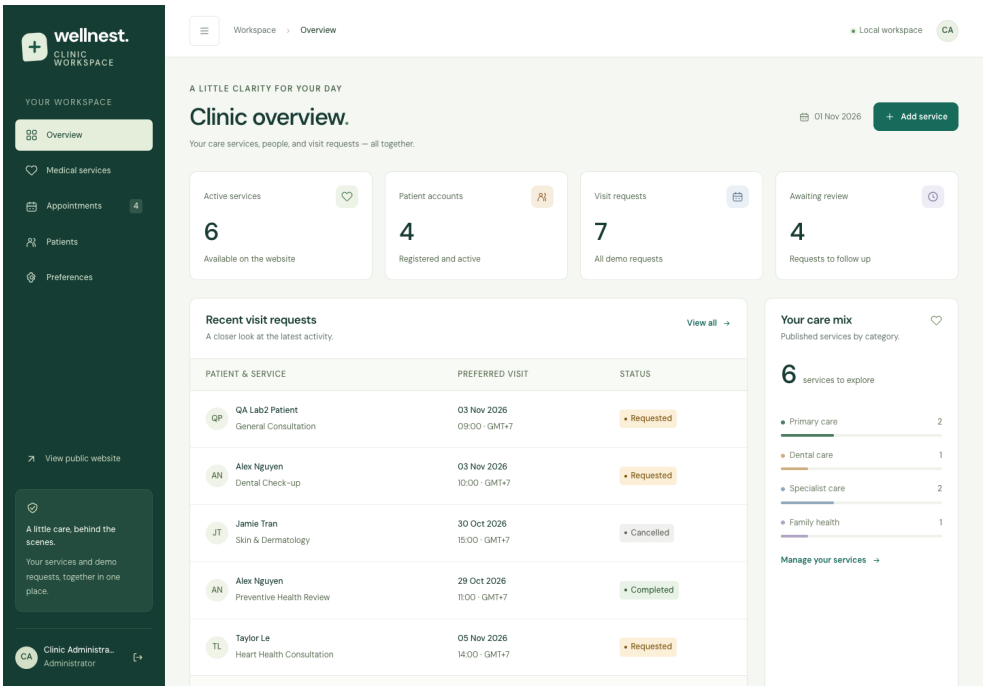


Figure 26. Overview computes service, patient and request metrics from MySQL.

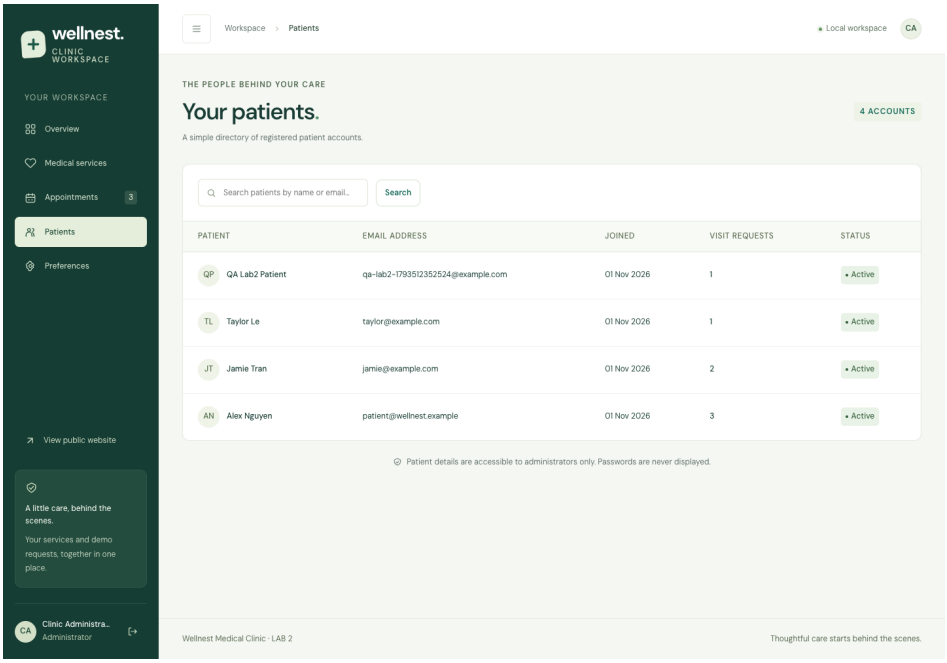


Figure 27. Patient directory is restricted to administrators and omits password hashes.

The workspace provides overview, service management, request status management, a searchable patient directory and preferences. Overview shows recent requests, publication distribution and audit activity. The directory joins patient accounts to request counts; it does not display passwords.

Service management supports search, publication filters and pagination. On narrow screens table rows become cards. The sidebar collapses on desktop and becomes an overlay drawer on mobile. Escape closes the drawer; focus is trapped inside the open drawer and closed navigation is inert.

16. Responsive administration evidence

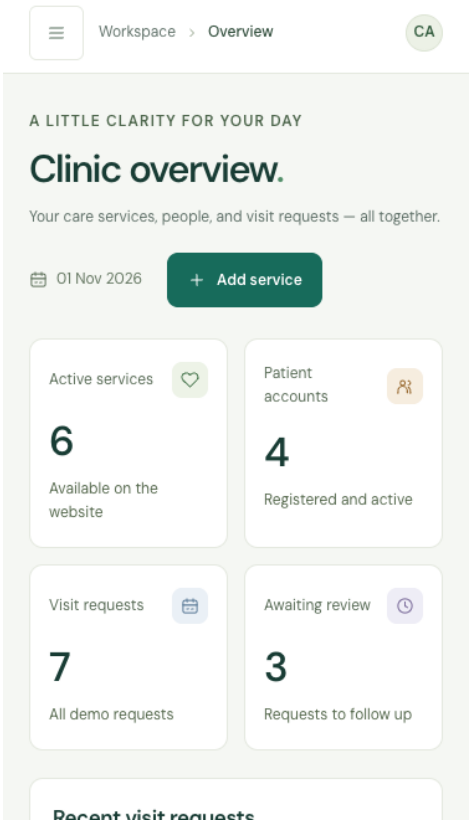


Figure 28. Overview at 375px.

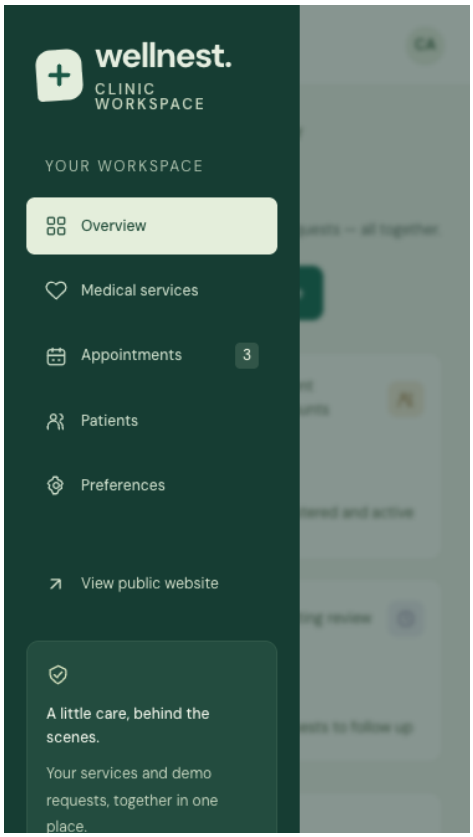


Figure 29. Mobile drawer with overlay.

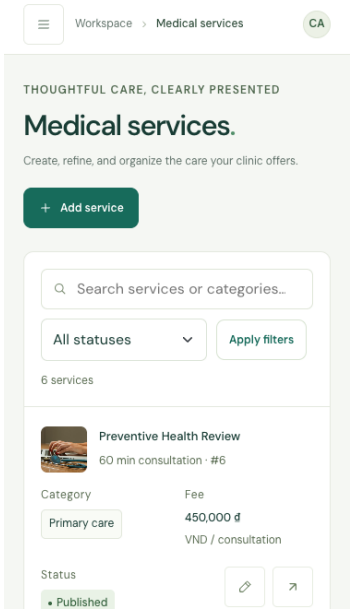


Figure 30. Service rows become mobile cards.

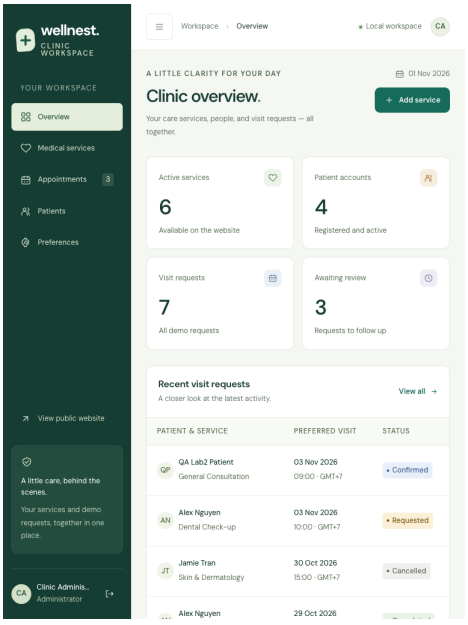


Figure 31. Tablet administration overview.

Small interface text was raised to at least 12px across the clinic stylesheets. Mobile form fields use 16px text to avoid automatic input zoom on common mobile browsers. Sidebar controls and service action buttons have at least a 44 × 44px target. Long names wrap and flex/grid children can shrink without forcing page overflow.

The overview request table was also converted to cards on mobile after accessibility QA identified a keyboard-inaccessible horizontal scroll area. Desktop tables remain available in their full form.

16.1 Responsive public and patient pages

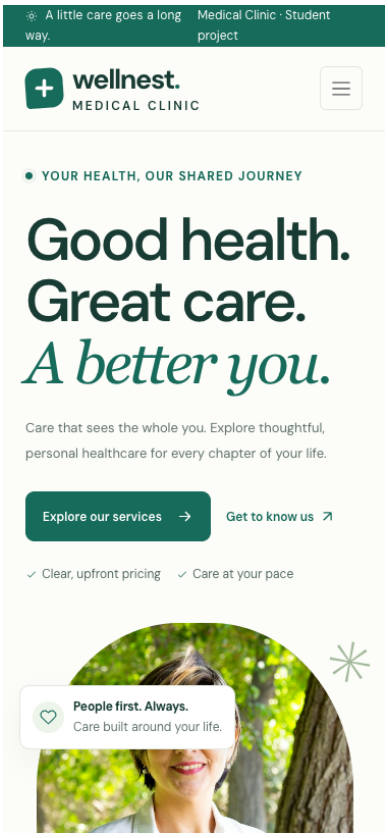


Figure 32. Public homepage on mobile.

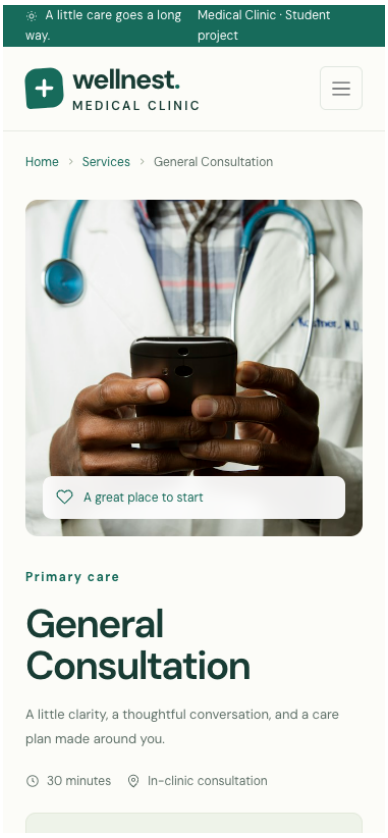


Figure 33. Service details on mobile.

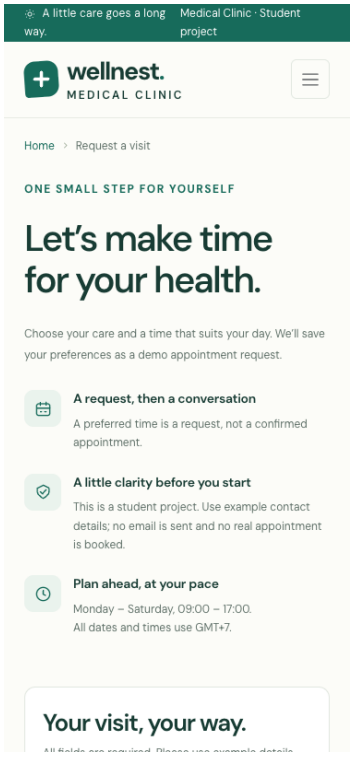


Figure 34. Stacked mobile request form.

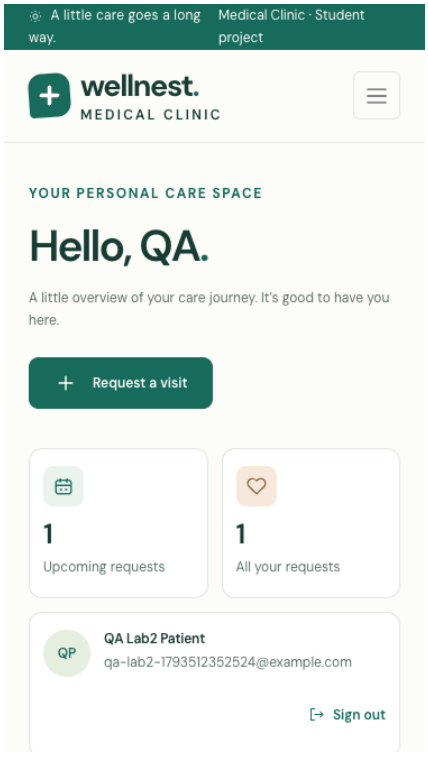


Figure 35. Patient dashboard on mobile.

The same breakpoint system supports the public site, authentication, patient pages and admin workspace. Required actions remain reachable as forms and cards stack vertically. Visit dates and fees wrap within their components rather than extending the document width.

16.2 Mobile registration and tested sizes

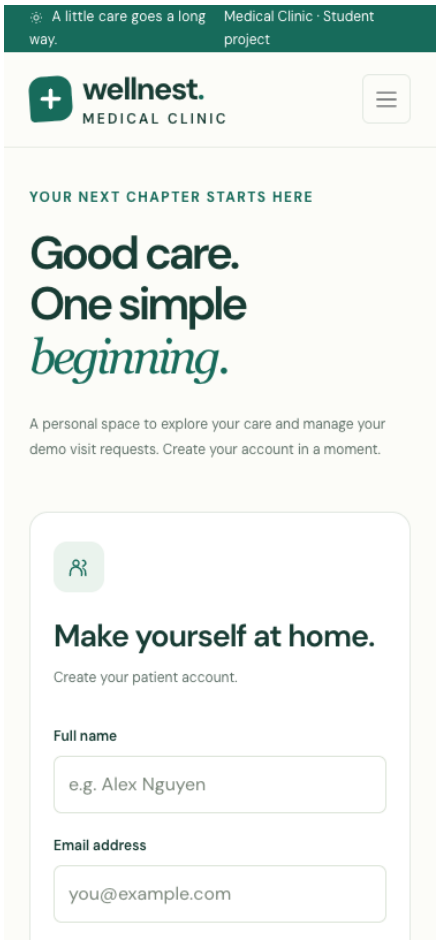


Figure 36. Registration retains readable labels and controls at 375px.

Actual tested viewports (CSS pixels): 320 × 700, 360 × 800, 375 × 667, 390 × 844, 414 × 896, 576 × 800, 768 × 1024, 820 × 1180, 844 × 390, 991 × 900, 992 × 900, 1024 × 768, 1280 × 800, 1366 × 768, 1920 × 1080, 2560 × 1440. The set includes a landscape phone, tablet widths, both sides of the 991/992px navigation breakpoint and a 2560px desktop.

The final browser run checked 208 page/viewport combinations across 16 sizes. It measured document and body widths against the viewport. Selected WCAG A/AA axe audits covered 26 page/viewport combinations at 375px and 1366px. These are explicit tested conditions, not a claim that every possible browser, screen or assistive technology has been exhaustively tested.

Manual screenshot review supplements automated overflow checks. Remaining browser coverage limit: the automated run used the installed desktop Chrome engine with emulated viewport sizes; physical iOS/Android devices and Safari/Firefox were not exercised by this suite.

17. Test method and final outcomes

Backend execution: 2026-11-01T12:50:45+07:00. Browser execution: 2026-11-01T05:52:32.524Z. All counts below come from the saved JSON results, rather than estimated test totals.

Verification layer	Observed result
Isolated backend	60 checks passed on a fresh temporary MySQL/MariaDB database.
Browser assertions	584 assertions passed, including functional, layout and accessibility assertions plus routine technical-error checks.
Responsive layout	208 page/viewport combinations without document/body horizontal overflow.
Accessibility	26 selected axe WCAG A/AA scans with zero reported violations.
JavaScript / assets	0 uncaught JavaScript errors; 0 failed local asset responses recorded.
Database failure	HTML and JSON missing-database scenarios returned generic HTTP 503 responses without sensitive details.
No-JavaScript fallback	Patient authentication, protected dashboard and native appointment submission passed.
Fresh SQL import	Six tables, four foreign keys, sample records and both documented demo passwords verified.

The browser suite runs against the actual Apache application, creates explicitly named QA accounts/services/requests, exercises separate patient/admin contexts and captures original PNG evidence. tools/cleanup_qa.php removes only the reserved QA records after capture. It retains the original sample clinic data.

The complete assertion labels and accessibility node details are retained in verification.json and backend-verification.json. error-handling-verification.json records the isolated outage checks. A failed check stops the run; the report generator refuses a failed verification result.

18. Validation and security test matrix

Input / action	Expected and observed behavior
SQL injection-shaped service ID	Rejected as an invalid identifier; no broad query or SQL error.
Markup in text / echoed content	Validation rejects prohibited markup; e() encodes displayed values.
Invalid fee, duration or category	HTTP 422 / field feedback; no service mutation.
Unapproved image path	Rejected; assets selected only from the local image library.
Malformed date, null byte, past date or Sunday	Field validation error; no calendar parser exception or saved request.
Email exceeding database limit	Rejected at 190 characters, consistent with VARCHAR(190).
Mismatched, short, oversized or null-byte password	Registration rejected; plaintext password is never inserted.
Duplicate email / submitted admin role	Duplicate rejected by unique key; new role forced to patient.
Forged CSRF token	HTTP 403; protected service/request mutation denied.
Patient direct admin GET/POST	HTTP 403 for every checked route and direct mutation attempt.
Guest protected request	HTML redirect to login / JSON HTTP 401.
Stale service or request version	Conflict rejected; newer changes preserved.
Referenced service deletion	HTTP 409; foreign-key relationships preserved.
Forged request fee / repeated submission	Server database fee retained; retry returns the original reference.
Logout and old-cookie replay	Protected access denied after session destruction.
Repeated incorrect sign-in	HTTP 429 after six failures within 15 minutes.

These are the basic security practices required for the laboratory plus selected integration protections. The checks document their tested scope; automated accessibility scans and validation assertions do not constitute a production security or medical-system certification.

19. Problems encountered and resolutions

Problem / implication	Implemented resolution and evidence
Lab 1 storage was insufficient for a MySQL-focused lab.	Central PDO MySQL connection, InnoDB schema and SQL import; isolated mysql-driver and FK assertions.
Old catalog interactions assumed IDs 1–6.	Generalized service IDs and counts; browser-created ID above six appears in search/save/detail flows.
Form email limit exceeded the database column.	Aligned appointment validation and maxlength to 190; regression with a syntactically valid long email.
Malformed calendar strings could reach the date parser.	Require ISO date structure before parsing; regression includes a null byte and oversized string.
Concurrent edits risked overwriting newer data.	Service and request version checks; stale update rejection assertions.
Deleting a referenced service could damage care history.	Restricting foreign key with actionable Draft guidance; direct POST deletion test.
Small responsive labels reduced readability.	Raised text below 12px; mobile inputs 16px; touch controls enlarged and long text allowed to wrap.
Mobile overview table failed keyboard accessibility.	Converted overview rows to mobile cards; repeated axe scans verify the observed failure is resolved.
Duplicate remote/local Bootstrap stylesheet requests.	Use the bundled local stylesheet only; local asset loading is checked.
Signed-out users lacked explicit completion feedback.	Added a signed-out session-ended notice on the login screen.
Source/runtime files could be exposed by the web server.	Apache and dev-router deny rules; real HTTP checks of config, SQL, Git, storage and tests.

Remaining scope: request times are preferences; there is no capacity scheduling, clinical record system, email delivery, payment gateway or production hosting in this lab. The code is retained for Lab 3 extension. No grade is claimed before instructor assessment.

20. Demonstration and submission instructions

Step	Demonstration and expected outcome
1 — Import / run	Start XAMPP Apache/MySQL; import database/lab2.sql; open /medical_clinic/. Show six tables in phpMyAdmin.
2 — Dynamic read	Open the public catalog and a detail page; compare its name/price with the MySQL service row.
3 — Registration	Register a new example email; try the same email again to show duplicate feedback; then sign in.
4 — Patient session	Navigate between My care and Request a visit; submit an illustrative valid request and record its reference.
5 — Role check	With the patient session open /admin/services.php; verify access is denied.
6 — Service CRUD	Sign in as admin; add an unreferenced service, verify public details, edit the fee, set Draft, then intentionally delete it.
7 — Synchronized status	Find the patient reference in admin, confirm it, reload patient history, cancel as its owner and reload admin.
8 — Cookie / logout	Collapse the desktop sidebar and reload; inspect Preferences; sign out and retry a protected URL.
9 — Responsive evidence	Inspect mobile/tablet/desktop PNGs and the saved viewport/accessibility results.

Demo login: admin@wellnest.example / WellnestAdmin!2026; patient@wellnest.example / WellnestPatient!2026. These are deliberate local grading accounts included in the sample SQL. Password hashes are verified by the backend tests.

Deliverables: complete PHP source; database/lab2.sql with schema and samples; forty original functional/UI/phpMyAdmin screenshots; this detailed PDF and editable DOCX; README instructions; verification JSON; and a ZIP under submission/. The package excludes private local configuration, live sessions, obsolete SQLite runtime data and Git metadata.

Reproduction: python3 tools/run_backend_tests.py; NODE_PATH=/tmp/wellnest-qa/node_modules node tests/browser.cjs; python3 tools/verify_error_handling.py; php tools/cleanup_qa.php. See README.md for dependency installation and platform-specific command paths. Build the report only after the verification results pass, then run tools/package_submission.py.

Appendix A. MySQL structure and password storage

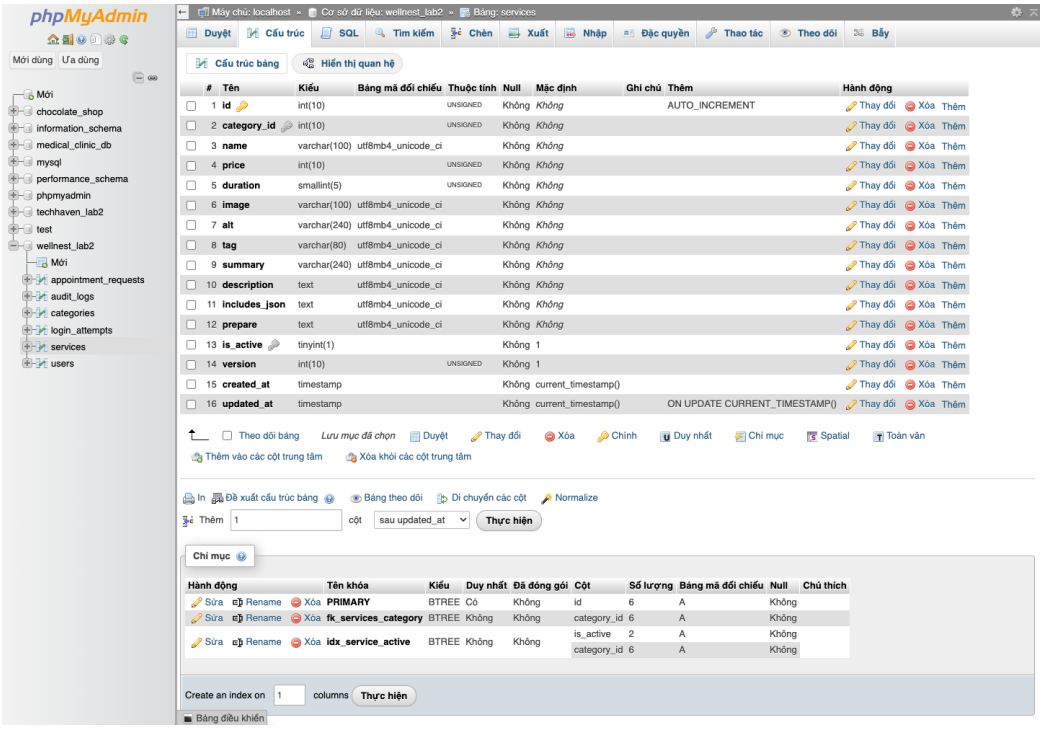


Figure 37. Actual phpMyAdmin view of the medical services table structure.

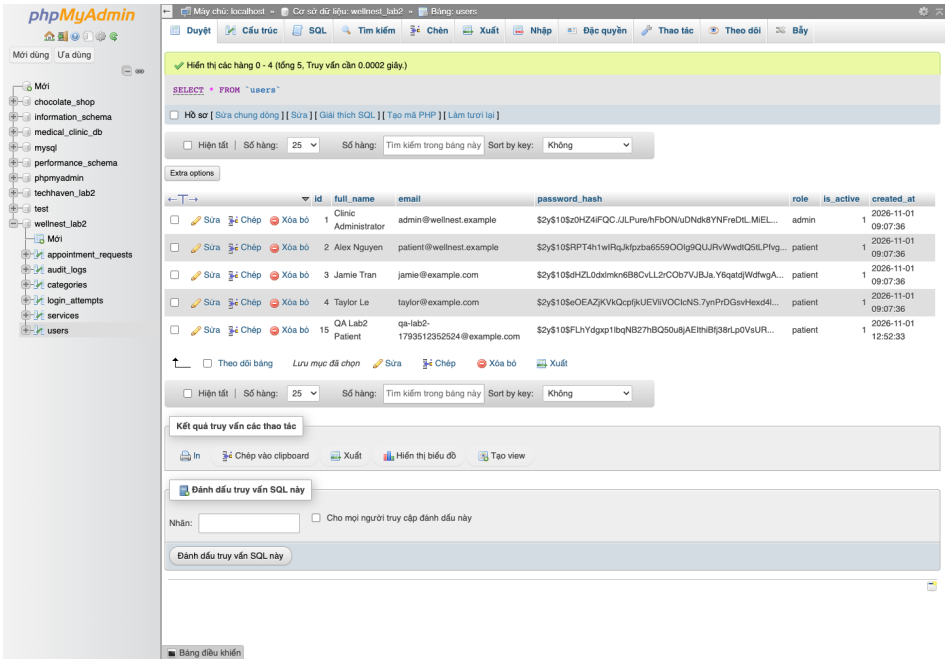


Figure 38. Actual account rows contain password hashes and role fields.

The account screenshot contains only illustrative seeded/test accounts. It demonstrates hashed storage; plaintext demo passwords appear only in the installation/demonstration instructions, not in the password_hash database column.

Appendix B. Complete public page and service list

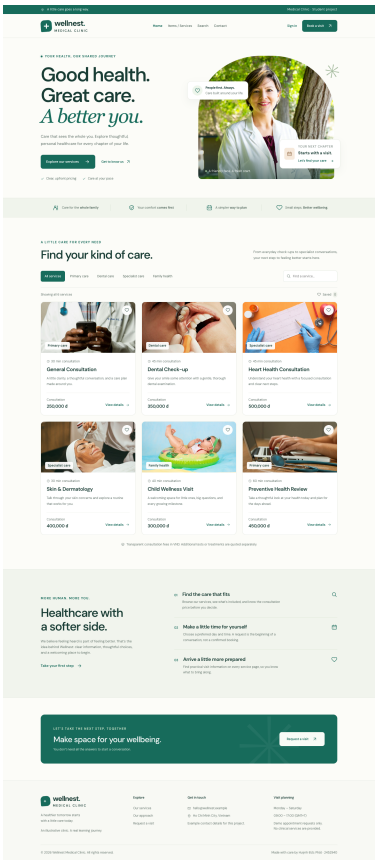


Figure 39. Full-page public capture. Open the original PNG for full-resolution section detail.

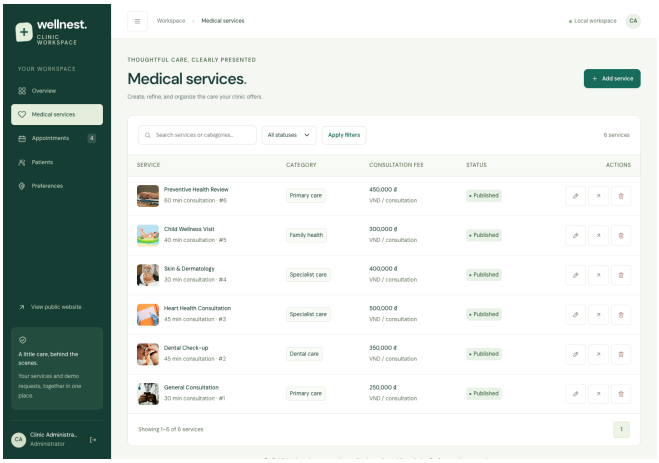


Figure 40. Administrator service list with filters, publication state and CRUD actions.

Original captures are stored individually in report/lab2/screenshots/. The full-page overview is reduced here to preserve its complete composition; the public hero and catalog are also shown at larger scale in figures 2–3. evidence-index.json maps every source PNG to its report section and caption.

Appendix C. Backend check details — 1–22

No.	Actual assertion	Result
1	Application uses the MySQL PDO driver	PASS
2	Six services and four supporting categories are seeded	PASS
3	User and role sample data exist	PASS
4	SQL demo password verifies: admin@wellnest.example	PASS
5	SQL demo password verifies: patient@wellnest.example	PASS
6	SQL import creates all four foreign key relationships	PASS
7	All six relational tables are present	PASS
8	Current user role is loaded from the database	PASS
9	Successful login rotates the session ID and CSRF token	PASS
10	Nested admin URLs resolve to the project root	PASS
11	Invalid service identifiers and SQL injection are rejected	PASS
12	Output encoding prevents raw HTML injection	PASS
13	External login redirect is rejected	PASS
14	Patient redirect cannot target the admin workspace	PASS
15	Registration accepts Unicode names and normalizes email	PASS
16	Passwords are hashed and verified securely	PASS
17	Registration rejects invalid email (16)	PASS
18	Registration rejects invalid full_name (17)	PASS
19	Registration rejects invalid password (18)	PASS
20	Registration rejects invalid password (19)	PASS
21	Registration rejects invalid password (20)	PASS
22	Registration rejects invalid password_confirmation (21)	PASS

Source: tests/backend.php. Runner: tools/run_backend_tests.py. Each run imports the supplied SQL into an isolated disposable database and removes that database in cleanup.

Appendix C. Backend check details — 23–44

No.	Actual assertion	Result
23	Registration rejects invalid consent (22)	PASS
24	Unique email prevents case-insensitive duplicate accounts	PASS
25	Valid service form passes server validation	PASS
26	Service validation rejects invalid price (25)	PASS
27	Service validation rejects invalid price (26)	PASS
28	Service validation rejects invalid duration (27)	PASS
29	Service validation rejects invalid category_id (28)	PASS
30	Service validation rejects invalid image (29)	PASS
31	Service validation rejects invalid name (30)	PASS
32	Service validation rejects invalid includes (31)	PASS
33	Service validation rejects invalid is_active (32)	PASS
34	Create stores a new MySQL service	PASS
35	Update changes the same record read by public pages	PASS
36	Optimistic version check prevents overwriting a newer edit	PASS
37	Drafts are hidden from public reads but remain available to admins	PASS
38	Intentional deletion removes an unreferenced service	PASS
39	Foreign keys preserve services referenced by appointment history	PASS
40	Valid appointment request passes server validation	PASS
41	Appointment rejects invalid preferred_date (40)	PASS
42	Appointment rejects invalid preferred_date (41)	PASS
43	Appointment rejects invalid email (42)	PASS
44	Appointment rejects invalid preferred_date (43)	PASS

Source: tests/backend.php. Runner: tools/run_backend_tests.py. Each run imports the supplied SQL into an isolated disposable database and removes that database in cleanup.

Appendix C. Backend check details — 45–60

No.	Actual assertion	Result
45	Appointment rejects invalid preferred_date (44)	PASS
46	Appointment rejects invalid preferred_date (45)	PASS
47	Appointment rejects invalid preferred_time (46)	PASS
48	Appointment rejects invalid service_id (47)	PASS
49	Appointment rejects invalid consent (48)	PASS
50	Appointment retry is idempotent	PASS
51	Appointment fee is read from the database, not client input	PASS
52	Service edits preserve historical request name and fee snapshots	PASS
53	A patient cannot alter another account’s request	PASS
54	Admin can move a requested appointment to confirmed	PASS
55	Stale appointment status updates are rejected	PASS
56	Closed appointments cannot return to an earlier status	PASS
57	Patients cannot confirm their own requests	PASS
58	Patients can cancel their own pending requests	PASS
59	Cancelled requests cannot be changed again	PASS
60	Mutating operations produce an audit trail	PASS

Source: tests/backend.php. Runner: tools/run_backend_tests.py. Each run imports the supplied SQL into an isolated disposable database and removes that database in cleanup.

Appendix D. Browser evidence and verification limits

The browser result contains 68 functional/environment assertions, 208 responsive assertions, 26 accessibility assertions and routine page-output checks. verification.json stores every assertion label and inspected viewport.

Page group	Covered routes
Public	public/index.php; pages/item_details.php?id=1.
Authentication	auth/login.php; auth/register.php; POST auth/logout.php.
Patient	pages/dashboard.php; pages/appointment.php.
Administrator	admin/index.php; services.php; service_form.php; service_delete.php; appointments.php; patients.php; preferences.php.

Acceptance evidence includes CRUD success and rejection paths, independent administrator/patient sessions, cookie persistence and expiry, direct role bypass attempts, lost-session access, old-cookie replay, no-JavaScript PHP submission, actual phpMyAdmin views and successful local asset requests.

Viewport tests establish lack of page-level horizontal overflow under the tested content and dimensions. They do not prove that every content string, zoom level, embedded webview or future browser will behave identically. Automated axe scans cover selected WCAG rules and cannot replace complete manual assistive-technology testing.

The report records observed passing results and resolved defects. The submission is prepared against all supplied Lab 2 requirements; final scoring is determined by the instructor. No “100% on all screens” or guaranteed numerical grade is asserted.